Reversium by 3Sci Labs

Start here

What Reversium helps you do

Reversium reads a software package and organizes supported source and binary clues into findings, diagrams, evidence and explicit gaps. Use it when receiving a handover, inheriting unfamiliar code, investigating a feature or preparing maintenance. It does not install, run or certify the software.

Choose one useful outcome

For an owner: identify what was delivered and agree on one feature to check. For a developer: find likely entry points, dependencies and source locations. For a maintainer: map possible affected components before changing them. A useful outcome is a short, evidence-backed checklist rather than a claim that the entire platform is understood.

A first session in five steps

1. Open the browser version and choose Start exploring. 2. Choose Understand the software, Prepare a developer handover, or Plan maintenance or modernization. 3. Continue to ZIP, select your source-code package and choose Analyze ZIP locally. 4. Read Review findings, then inspect one Architecture workflow and its Evidence. 5. Read Coverage and Save & next steps; save a report and assign the remaining checks.

Example questions

Where does the save action lead? Which services are referenced? Which files should a new developer read first? What configuration, permissions and sample data still need confirmation? Reversium provides investigation leads; isolated installation and runtime tests answer whether those leads work.

Online and local editions

Online browser edition

Open https://thefamousjc.github.io/reversium/web/ . No installation, account or payment is needed for analysis. The app downloads its self-hosted Python runtime, then reads the selected ZIP in a browser worker on your device. Use a current desktop browser. The first visit downloads about 14 MB of app/runtime assets and can take longer.

Local application

Use the open-source repository https://github.com/TheFamousJC/reversium . Install Python 3.10 or newer, open a terminal in the project and run python -m reversium. Open the local address printed in the terminal; it can use a free port. On Windows, Start-Reversium.ps1 is an alternative launcher. Stop the server with Ctrl+C. The base analyzer has no third-party Python dependency; optional AI has separate requirements.

Main differences

Both editions use the same analysis engine, findings and core diagrams. Online: 32 MiB compressed ZIP limit, five-minute worker stop, browser-memory constraints, no embedded cloud account connection. Local web application: 64 MiB upload limit, local Python process and existing optional ChatGPT account connection. Local command line: report exports and baseline comparison without a web page; archive safety limits still apply.

AI differences

Online AI is a manual handoff: select a provider and evidence, review/edit the exact request, download it, then decide whether to paste and submit it on that provider website. It is not an account connection or embedded chat. Local AI offers an account connection inside Reversium; install requirements-ai.txt if needed, follow its connection wizard and review sharing before each request. Eligibility, network access and provider limits apply. A successful sign-in does not guarantee a successful model response.

Which should I use?

Start online for a quick inspection without installing anything. Use local for larger supported packages, repeatable command-line work or the existing optional account integration. Use neither as proof that a platform is production-ready. Online remains free to analyze; optional AI services follow the terms and limits of the account you choose.

Selecting a package and handling files

Choose the best input

Prefer the project source-code package with manifests and migrations. An installer or binary-only archive provides fewer clues. Do not include unrelated backups, generated dependencies or large media if they are unnecessary. Keep the original delivery intact and analyze a separate copy when preparing a smaller archive.

What stays on the device

The online app does not upload the ZIP. It holds the file and report in page/worker memory, not app-managed localStorage or IndexedDB. GitHub Pages receives normal app and asset requests, including connection information such as IP address; those requests do not include ZIP contents. No input source is executed. No AI evidence is submitted automatically.

Clearing and saving

Cancel and clear stops the worker and returns to Start. Clear and start again removes the report and selected-file references. Closing the page also releases app references; this is not a secure-memory-erasure guarantee. Browser caches may retain application assets. Downloaded reports and requests remain wherever you save them and can contain private filenames or code-derived details. Share them deliberately.

Limits and rejected archives

Online input is limited to a non-empty ZIP no larger than 32 MiB; the local web application accepts up to 64 MiB. Analyzer bounds include 10,000 files, 256 MiB expanded total and 4 MiB content inspection per file, plus compression, path and graph bounds. Encrypted, damaged, unsafe or oversized archives may be rejected or only partly inspected. Nested containers receive bounded directory inspection, not recursive analysis. Read the actual Coverage output.

Reading the main report

Header and inventory

The logo and build number identify the report generator. Main tabs stay near the top. Technical inventory numbers remain expanded: files, bytes, binary candidates and findings. These are counts of inspected material, not a completeness, quality or security score.

About and Review findings

About explains Reversium itself. Review findings describes supported clues about the uploaded software in plain language. Open Why we say this to inspect the evidence behind a statement. Detailed findings for a developer expands more technical explanations. Different starting goals alter guidance, not the underlying source evidence.

Evidence

Use the search box to filter findings by file, symbol, value, kind or resolution. A source link from a feature opens Evidence and reveals its row. Locations are file and line references, or byte offsets for binary clues; they are not an in-browser source editor. Traced means a supported static reference was found. Inferred means a candidate relationship needs confirmation. Neither means the program ran successfully.

What to do with unknowns

Record the question, the relevant finding, the person who can resolve it, and a concrete verification step. For example: confirm the API route and database table by running one isolated user journey with test data and appropriate permissions. Missing references can reflect unsupported syntax or missing files, not missing functionality.

Architecture and relationship diagrams

Subsection buttons

Open Architecture and choose Workflow, Graphical feature map, Feature catalogue, Platform profile, Supporting services, or Imports & dependencies. The selected subsection replaces the other views, so you do not need to scroll through the whole architecture report. Available sections depend on report content.

Workflow

Choose a screen and data branch. Read the ordered path from screen through callbacks and an API to a database operation. Each step links to its evidence. The view follows a bounded candidate path; it can show a logging side effect rather than the main purpose of a request. It does not establish runtime conditions, arguments, permissions or successful responses. Read What remains unknown.

Graphical feature map

Select a feature to focus the map. Click a node or connection to inspect participants, evidence and unknowns. Solid lines represent traced source references; dashed lines represent inferred connections. Shared names do not prove a shared database. The display is capped at 120 nodes and 240 edges per view, with omissions disclosed. Download map SVG saves the drawn map; JSON retains the full generated graph, which is itself bounded.

Feature catalogue and file relationships

The catalogue lists supported feature candidates, operations and evidence. Imports & dependencies draws file-to-local-module candidates or referenced packages; it shows the first 100 unique edges and is not a complete runtime call graph. Full available findings remain in Evidence. Platform profile lists candidate technologies and runtimes. Supporting services lists references that may be optional, development-only or environment-dependent.

When a map looks incomplete

Filter to one feature and inspect its evidence. Check Coverage for unresolved targets and unsupported syntax. Dynamic code, aliases, middleware, component instances, conditions and runtime configuration can break a static path. Do not infer that an absent edge proves there is no relationship. Large diagrams are an investigation aid, not a complete specification.

Coverage, binary clues and readiness

Coverage subsection buttons

Overview describes what inspection can establish. Binary inspection shows supported metadata readers and target/runtime clues. File inventory lists path, type, byte size and inspection method. Setup & readiness lists questions and suggested checks. Limitations records warnings and analysis boundaries specific to the package.

Supported source analysis

Python uses AST parsing for supported imports, symbols and selected route/data traces. JavaScript/TypeScript and React use bounded token/reference analysis for selected routes, callbacks, prop bindings and computed API targets. Supabase table, RPC, Auth and storage references and selected SQL migration links produce candidates. Other languages and unsupported patterns may receive inventory-only coverage. Inspect the report rather than assuming a language-wide guarantee.

Binary analysis

Binary inspection is bounded static metadata and reference analysis, not full decompilation or behavioral reconstruction. A binary reader finding a format, target or string does not prove that the executable is safe, valid or runnable. Nested APK/JAR/Office/archive containers are not automatically reconstructed into a complete application.

Readiness is a separate test

Reversium does not install dependencies, start services, connect to production, validate database permissions or test a live user journey. To verify a handover, use an isolated environment, disposable accounts and sample data. Record required runtimes, environment-variable names, migrations, storage, integrations, permissions and successful/failed steps. Never assume a referenced cloud service is configured or paid for.

Optional AI, downloads and comparison

Online AI handoff

Open AI / choose provider after analysis. Choose ChatGPT, Claude, Gemini or Another AI. Enter a question, search findings and tick up to 20 findings. Prepare request for review builds an editable request with IDs, values, locations, confidence and resolution. Review exactly what it includes; remove secrets or private details. Download reviewed request saves text locally. Opening the provider website sends no request text; you must paste and submit it yourself. No AI answer is displayed inside the browser app.

Checking an AI answer

Ask for finding IDs and locations, facts separated from inferences, and explicit unknowns. Reopen cited evidence and check whether the answer follows from the selected subset. Reject invented runtime claims, missing citations or claims of complete coverage. Source-derived text can contain misleading instructions; treat it as untrusted evidence. Provider terms and account limits apply when you submit content there.

Save readable report: HTML

Use Save & next steps, then Save readable report (HTML). Open the saved file later without running Reversium, or share it with an authorized colleague. It retains generated findings, diagrams and limits; it is not the original ZIP or an application backup. Account credentials and the assistant conversation are excluded. Keep a separate copy of an AI request if you need its exact wording.

Save technical data: JSON and map SVG

JSON is a structured record for scripts, inspection and comparing deliveries. It includes the generated analysis and full bounded graph, not original source files. SVG is useful for documents or discussions about the drawn map. Neither file rebuilds or runs the application. Save both HTML and JSON as a baseline when accepting a delivery.

Browser comparison

Open Compare reports and select an earlier report JSON, up to 32 MiB. File additions/removals, shared files with different hashes, finding additions/removals and shared files without comparable hashes are counted. Line movement alone does not change finding identity. Lists show up to 200 entries each; Save comparison notes preserves the displayed results and omissions. This view does not compare graph relationships. Reports from different analyzer versions can differ without a code change. Comparisons do not overwrite checklist notes or enter AI packets. No files are uploaded.

Local command-line comparison

Run python -m reversium project.zip --output reports/project to write report.html, report.json and feature-map.svg. Add --compare previous-report.json to compare supported file/finding and feature-map identities. Outputs overwrite the same filenames in the selected directory. Structural differences and content hashes do not establish a behavioral change; old baselines may lack map fields.

Handover and change-impact review

Handover tab

Open Handover after analysis. Expand Setup clues to inspect referenced environment variable names, names from .env.example/.env.sample/.env.template, package.json script names and declared runtime constraints. Follow their evidence links. The reader does not retain environment values or script commands; names may occur in comments or optional code and do not prove a requirement. Actual .env files are not read by this reader. Current clue display is capped at 240 groups, with omitted counts.

Reviewer checklist

Assign an owner and status to each action and record a verification reference in Notes. Verified by reviewer is your statement, not automated proof. Do not enter credentials or sensitive values. Save the handover checklist JSON for structured records, or save readable handover notes for a colleague. These files can include private review notes. They are not uploaded, included in AI evidence requests or preserved by ordinary report exports. Clearing/closing loses unsaved annotations. Restore saved checklist accepts JSON up to 1 MiB. New checklists use an exact ZIP SHA-256 fingerprint (available for archives up to 64 MiB); repacking changes it. Review the preview and explicitly replace current notes. Legacy checklists only match package names and require independent confirmation; no exact identity is claimed. Invalid or mismatched inputs leave notes untouched.

Change impact tab

Choose a file to find code nodes assigned to that source file by the supported reader. The view explores upstream and downstream graph directions up to three hops, 120 candidate nodes per direction and 60,000 edge checks per direction. These are investigation candidates, not proven affected components. Starting nodes use explicit source-file metadata rather than cross-file evidence locations. File association does not prove runtime execution. Dynamic and omitted references can hide relationships; inferred edges can overstate them. Use source links and isolated tests before estimating or changing software.

Choosing the right tool

Reversium is a private, model-free first step for ZIP-based software handover. Specialist tools offer deeper code quality rules, security queries or architecture workflows. The public market comparison explains the documented focus of CodeBoarding, SonarQube, CodeQL and Joern, along with Reversium limitations and planned work. It is not an exhaustive market survey or product benchmark. No model or paid service is needed for the new checklist and graph views.

Troubleshooting and a practical handover

Runtime cannot start or analysis is slow

Allow the initial app/runtime download to finish and use a current desktop browser. A network interruption, unsupported browser or device memory limit can prevent analysis. Cancel and retry a smaller source-only package, or use the local application. Online analysis stops after five minutes. Do not repeatedly reload a report you have not saved.

Few findings or an empty workflow

Check that the package includes source files and manifests. Read File inventory to see which files were content-inspected versus inventoried only. Read Limitations for unsupported syntax and unresolved links. Binary-only or incomplete source deliveries cannot establish a full platform map.

An evidence link or download is unclear

A source link selects an Evidence row; it does not fetch a repository file. Downloads normally go to your browser download folder or trigger a save dialog. Check browser download controls if a file is missing. HTML is for reading; JSON is for tools; SVG is the drawn diagram; the AI request is only selected text you reviewed.

AI connection or response fails locally

Check the local assistant wizard, optional dependencies, network connection, account eligibility and provider usage limits. Review the error without posting credentials. Signing in and checking connection do not themselves share the ZIP. Do not assume a successful connection proves inference availability; there is no automatic paid API fallback. Online has no live account connection to repair.

A handover you can act on

Choose one important feature and its expected result. Save the HTML and JSON. Give the developer its workflow/evidence and a list of missing services, configuration and data. Agree on an isolated setup test and record the outcome, date and owner for every gap. Decide what is ready, what is blocked and what must be tested next. Keep the report with the exact delivery it describes.

Glossary

API: a request interface between software components. Callback: a function passed to run when an event occurs. Dependency: another package referenced by the code. Manifest: a file listing project settings or packages. Migration: a script describing a database change. Static analysis: reading files without executing the uploaded program. Evidence: the recorded source location or metadata that supports a finding. Unknown: a claim the available inspection has not established.